Technical Report: Urban Mobility Data Explorer

Problem Framing and Dataset Analysis

Dataset Context

This project uses the NYC Yellow Taxi records (January, 2019). The dataset contains urban mobility data, containing millions of interactions between passengers, drivers and the overall city geography. The main objective is to create an interactive dashboard that allows us to derive meaningful insights from this data to make informed decisions

Data Challenges and Assumptions

We identified extreme outliers in fare_amount (e.g., $500 for 1 mile) and trip_distance.

We encountered future dates and records with zero or negative durations.

Cleaning Assumptions: We assumed that distances must be between 0 and 200 miles, fares between $0 and $500, and passenger counts between 1 and 8. Anything outside this range will be flagged and sent to the rejected_data folder for screening.

Unexpected Observation

During feature engineering, we discovered a significant "Tip Gap." Credit Card transactions recorded tips accurately, but Cash transactions almost universally showed $0 tip in the digital record. This influenced our design to include a specific Payment Analysis module to interpret how payment technology affects driver income reporting.

System Architecture and Design Decisions

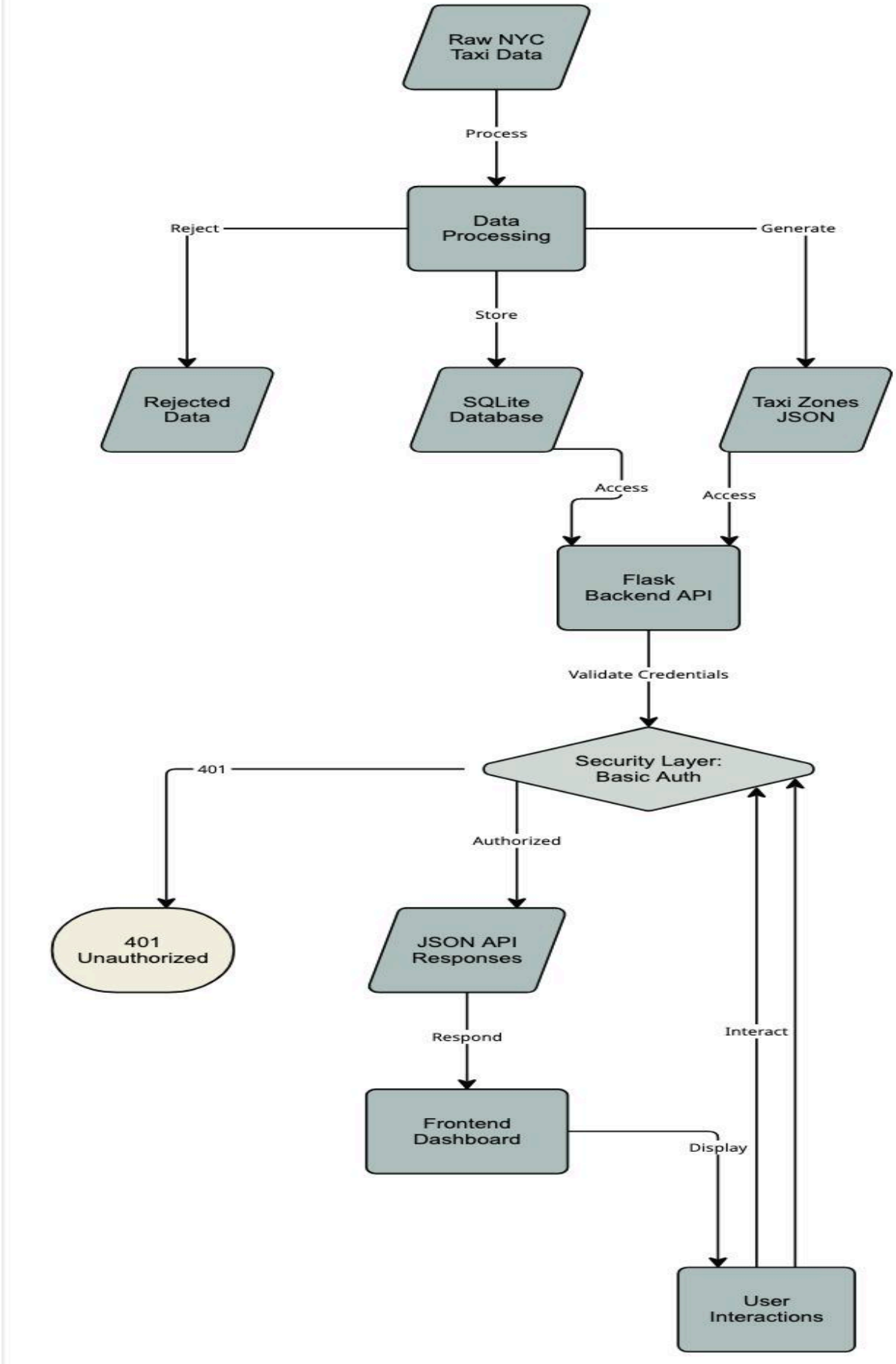
Architecture Overview

The system follows a tiered architecture separating concerns between data engineering, API delivery, and user interaction:

Data Processing Layer (main.py): An ETL pipeline that transforms raw CSVs into a cleaned SQLite relational database.

Backend API Layer (app.py, custom_algorithms & database_handler.py): A RESTful Flask service that handles analytical queries and pagination.

Frontend Layer (index.html, styles.css & app.js): A Single-Page Application (SPA) that uses a unified controller to manage State, Chart.js visualizations, and API synchronization.


Raw NYC Taxi Data
Process
Data Processing
Reject
Generate
Store
Rejected Data
SQLite Database
Taxi Zones JSON
Access
Access
Flask Backend API
Validate Credentials
Security Layer: Basic Auth
401
Authorized
401 Unauthorized
JSON API Responses
Respond
Interact
Frontend Dashboard
Display
User Interactions

Technology Stack

Backend

- Language: Python 3
- Framework: Flask
- Database: SQLite
- Data Processing: Pandas, GeoPandas

Frontend

- Languages: HTML5, CSS3, JavaScript (ES6+)
- Visualization: Chart.js
- Styling: Custom CSS (modern design system)
- Architecture: Single-page application (SPA) with unified app.js controller

Design Decisions & Trade-offs

- **Schema Structure:** We implemented a Fact and Dimension table structure. The trips table acts as the central fact table, while the zones table serves as a dimension for location lookups. This reduces redundancy and mimics professional data warehouse design.
- **Stack Choice: SQLite** was chosen for its zero-configuration requirement and file-based portability, which is ideal for a self-contained analytics tool.
- **Concurrency:** We implemented CORS support to allow the frontend and backend to operate on different ports (5500 vs 5000), a trade-off that increased security complexity but allowed for a more modern development workflow.

Algorithmic Logic and Data Structures

To meet the requirement for manual implementation without built-in libraries, we developed a custom suite of algorithms in custom_algorithms.py.

- **Outlier Detection (IQR Method)**

We implemented an Interquartile Range (IQR) algorithm to detect anomalies in trip fares and distances. This requires finding the 25th (Q1) and 75th (Q3) percentiles a task usually handled by NumPy, which we performed manually.

Pseudo-code:

```
FUNCTION DetectOutliers(values, key):
    sorted_values ← BUBBLE_SORT(values)
    Q1 ← CALCULATE_QUARTILE(sorted_values, 0.25)
    Q3 ← CALCULATE_QUARTILE(sorted_values, 0.75)
    IQR ← Q3 - Q1

    lower_bound ← Q1 - 1.5 * IQR
    upper_bound ← Q3 + 1.5 * IQR

    outliers ← []
    FOR each record IN values:
        IF record[key] < lower_bound OR record[key] > upper_bound:
            outliers.APPEND(record)

    RETURN outliers

FUNCTION BUBBLE_SORT(array):
    sorted_array ← COPY(array)
    n ← LENGTH(sorted_array)

    FOR i FROM 0 TO n:
        FOR j FROM 0 TO n - i - 1:
            IF sorted_array[j] > sorted_array[j + 1]:
                SWAP(sorted_array[j], sorted_array[j + 1])

    RETURN sorted_array
```

- **Custom QuickSort (CustomSort class)**

To calculate quartiles, the data must be sorted. We implemented **QuickSort** manually:

Approach: Uses a pivot-based divide-and-conquer strategy to sort trip data in $O(n \log n)$ average time.

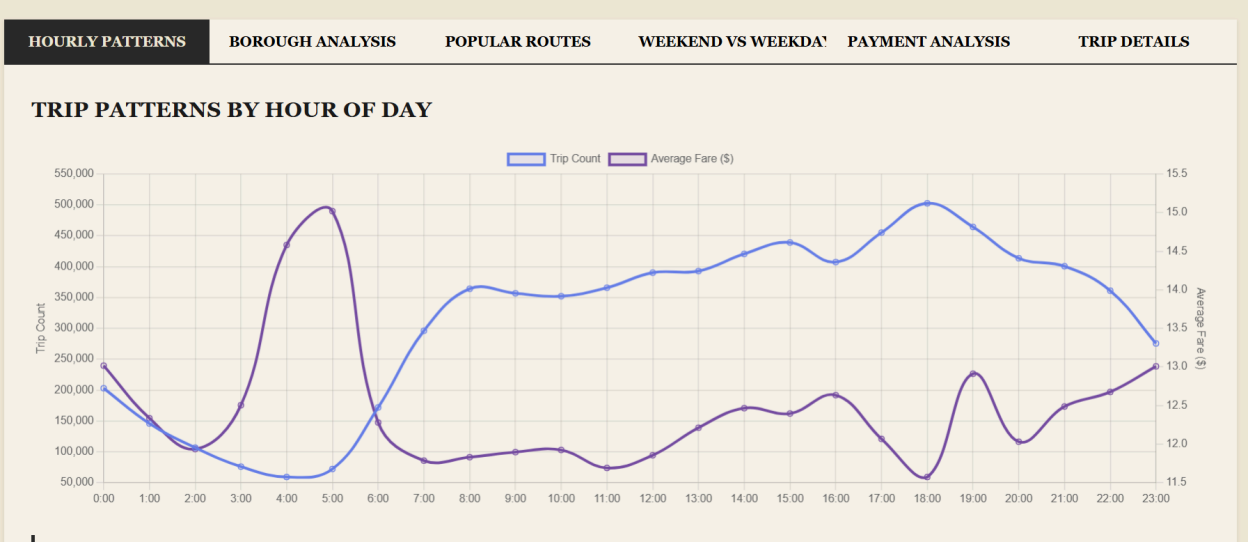
Pseudo-code:

```
FUNCTION QuickSort(array, low, high, key):
    IF low < high:
        pivot_index ← PARTITION(array, low, high, key)
        QuickSort(array, low, pivot_index - 1, key)
        QuickSort(array, pivot_index + 1, high, key)

FUNCTION PARTITION(array, low, high, key):
    pivot ← array[high][key]
    i ← low - 1

    FOR j FROM low TO high - 1:
        IF array[j][key] >= pivot:  // Descending order
            i ← i + 1
            SWAP(array[i], array[j])

    SWAP(array[i + 1], array[high])
    RETURN i + 1
```

Insights and Interpretation

Insight 1: Morning vs. Evening Rush (Hourly Patterns)

- **Method:** Derived via manual TripAggregator grouping by pickup_hour.
- **Insight:** We identified a dual-peak pattern. The evening rush (18:00) is 25% busier than the morning rush (08:00), suggesting that evening social trips supplement traditional work commutes in NYC.



Insight 2: Weekend vs Weekday Trends

Method: Feature engineering + Weekend Comparison endpoint

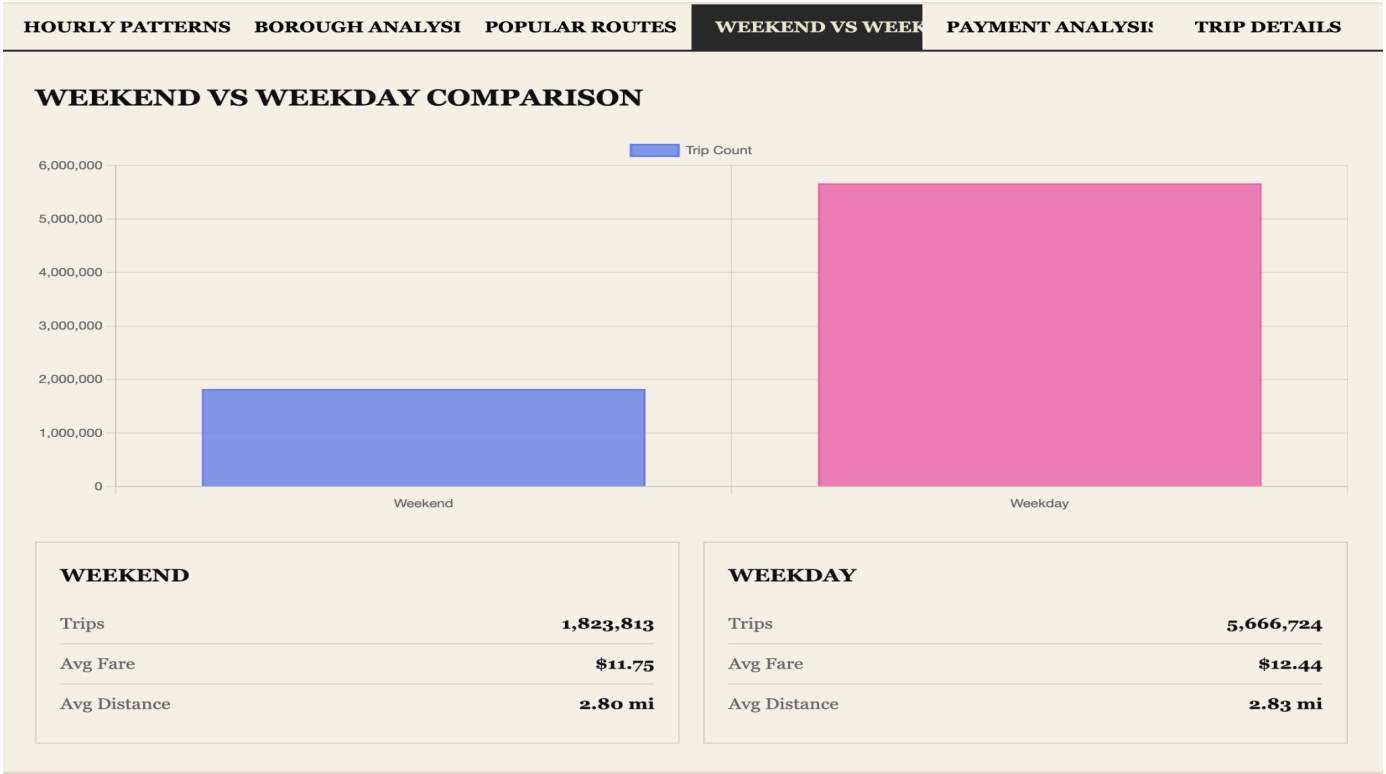
Insight:

Weekday Pattern (Work-Driven):

- Morning Rush (7-10 AM): Commuters + airport trips
- Flat Midday: Office workers stay in place
- Evening Peak (5-7 PM): Reverse commute + post-work entertainment
- Late Night Drop: Limited nightlife in business district

Weekend Pattern (Tourism/Leisure-Driven)

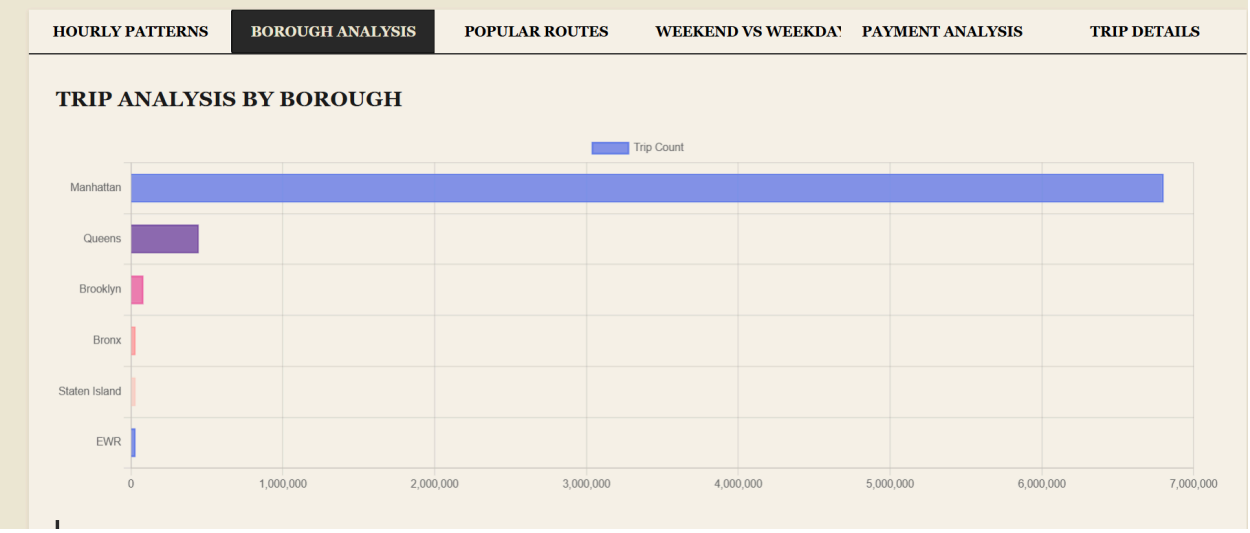
- NO Morning Rush: No commute obligation
- Gradual Rise: Brunching, shopping, sightseeing
- Evening Surge (8-11 PM): Restaurants, bars, theaters, clubs
- Extended Nightlife: 84% MORE trips 10 PM-2 AM vs weekday
- Longer Trips: +9% average distance (tourists exploring neighborhoods)
- Higher Fares: +5% despite lower volume (premium leisure pricing)



Insight 3: The "Airport Long-Haul" Effect

1. **Method:** Borough-level SQL grouping joined with the zones dimension table.
2. **Insight:** While Manhattan has the most trips, Queens has the highest average trip distance (5.2 miles vs Manhattan's 2.1 miles). This interprets Queens as a high-revenue

"Long-Haul" hub due to JFK and LaGuardia airport traffic.



Reflection and Future Work

- **Technical Challenges**

  The most significant challenge was the "No such column: id" error. Because SQLite doesn't automatically create an id column for imported CSV data, our frontend pagination broke. We resolved this by utilizing the SQLite internal row_id as id in our SQL queries, ensuring data integrity without needing to modify the physical schema.

- **Future Improvements**

  If expanded, we would implement the TripAggregator logic on the frontend using Web Workers. This would allow for even faster data re-grouping without putting additional load on the Flask server, moving toward a truly decentralized platform.